[ADR] Data redundancy strategy

Status

Proposed by @Ulad Palinski

Stakeholders: @Sergey Poluyan

Outcome

Status: implemented

Note: the values are set on cluster level, our Dragon 1 cluster has a 16/48 scheme, each new cluster has to specify it during cluster setup

Context

At the moment DDC uses a simple replication to achieve data redundancy. In order to achieve desired level of durability, storage makes N copies (e.g. 5). This yields an expansion factor of 5x, or 500%. Then this duplicated data (x5) needs to be stored on the network, using bandwidth.

Thus, higher level of durability results in more storage and bandwidth for a fixed amount of data.

So, the cost of durability is:

- higher storage price
- higher bandwidth
- lower network storage capacity (total size / replication factor)

Decision

Erasure coding

As an alternative to simple replication, **erasure codes** provide a much more efficient
method to achieve data redundancy.

Erasure codes are an encoding scheme for increasing data durability. This technique are widely used by both distributed (e.g. AWS S3, Ceph) and peer-to-peer storage systems (e.g. Storj, mentioned in Filecoin but not clear is it working or not).

An erasure code is often described by two numbers, **K** and **N**. If a block of data is encoded with a **(K, N)** erasure code, there are **N** total generated erasure shares, where
only any **K** of them are required to recover the original block of data. If a block of data
is **S** bytes, each of the **N** erasure shares is roughly **S/K** bytes. Each erasure share is unique.

More interestingly, erasure codes can increase durability without increasing the expansion factor. The durability of a **(K = 20, N = 40)** erasure code is better than a **(K = 10, N =
20)** erasure code, even though the expansion factor (2x) is the same for both. This is because the risk is spread across more nodes in the **(K = 20, N = 40)** case.

Durability comparison (erasure coding vs replication)

Erasure coding

k	n	Exp. factor	$P(D \mid p = 10\%)$
2	4	2	99.2073668132746 16%
4	8	2	99.858868985411326%
8	16	2	99.995462406878260%
16	32	2	99.999994620652776%
20	40	2	99.999999807694154%
32	64	2	99.999999999990544%

Replication

k	n	Exp. factor	$P(D \mid p = 10\%)$
1	1	1	90.483741803595962%
1	2	2	98.247690369357827%
1	3	3	99.640050681691051%
1	10	10	99.999888857452166%
1	16	16	99.999999998036174%

Expansion factor

The expansion factor still plays a big role in durability, as seen in the following table:

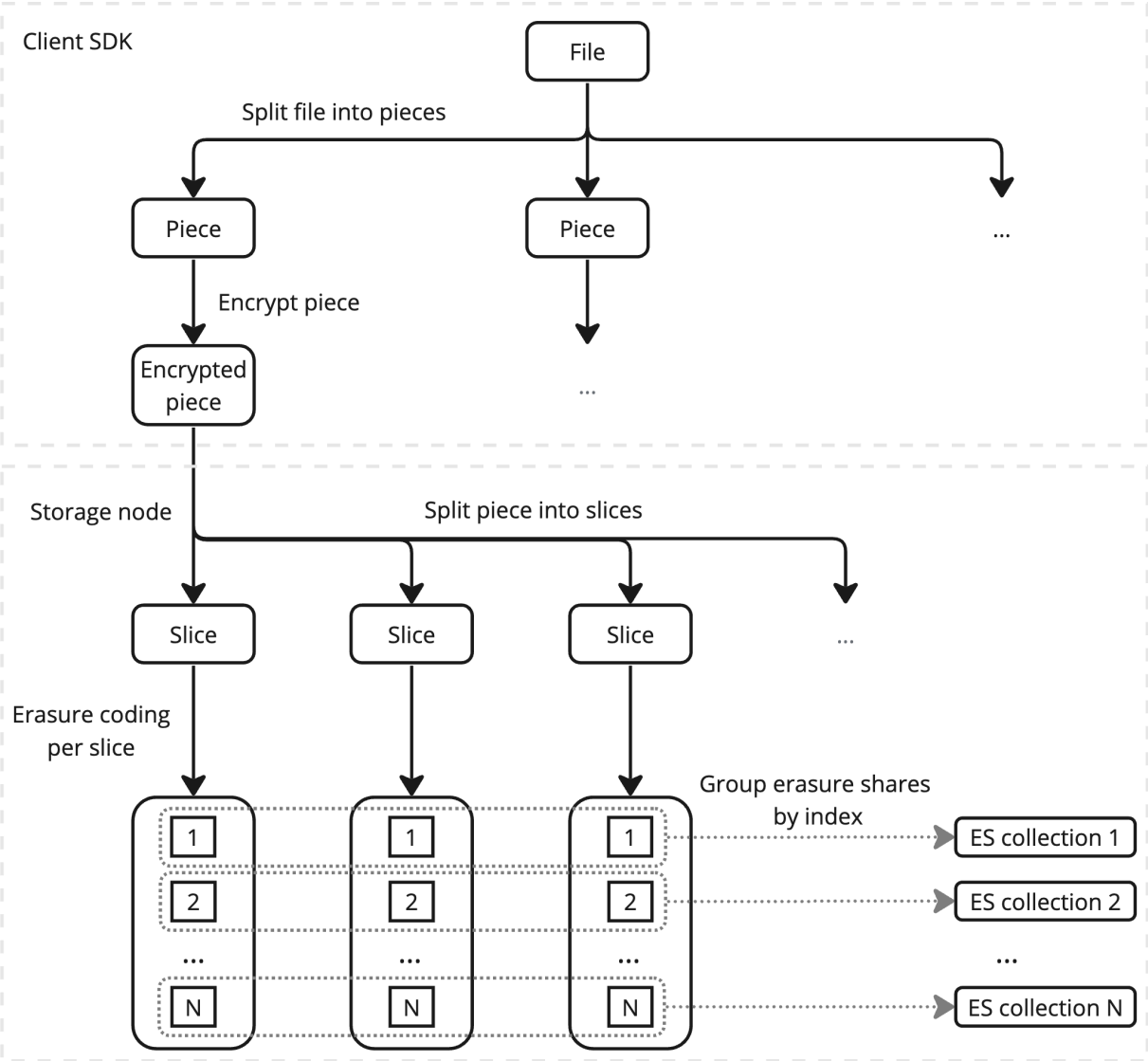
k	n	Exp. factor	$P(D \mid p = 10\%)$
4	6	1.5	97.688471224736705%
4	12	3	99.999514117129605%
20	30	1.5	99.970766304935266%
20	50	2.5	99.999999999999548%
100	150	1.5	99.999999999973570%

Data organisation

Piece is the main data concept in the storage (in DAG API node is a piece). Its size can be limited to ~5 MB.

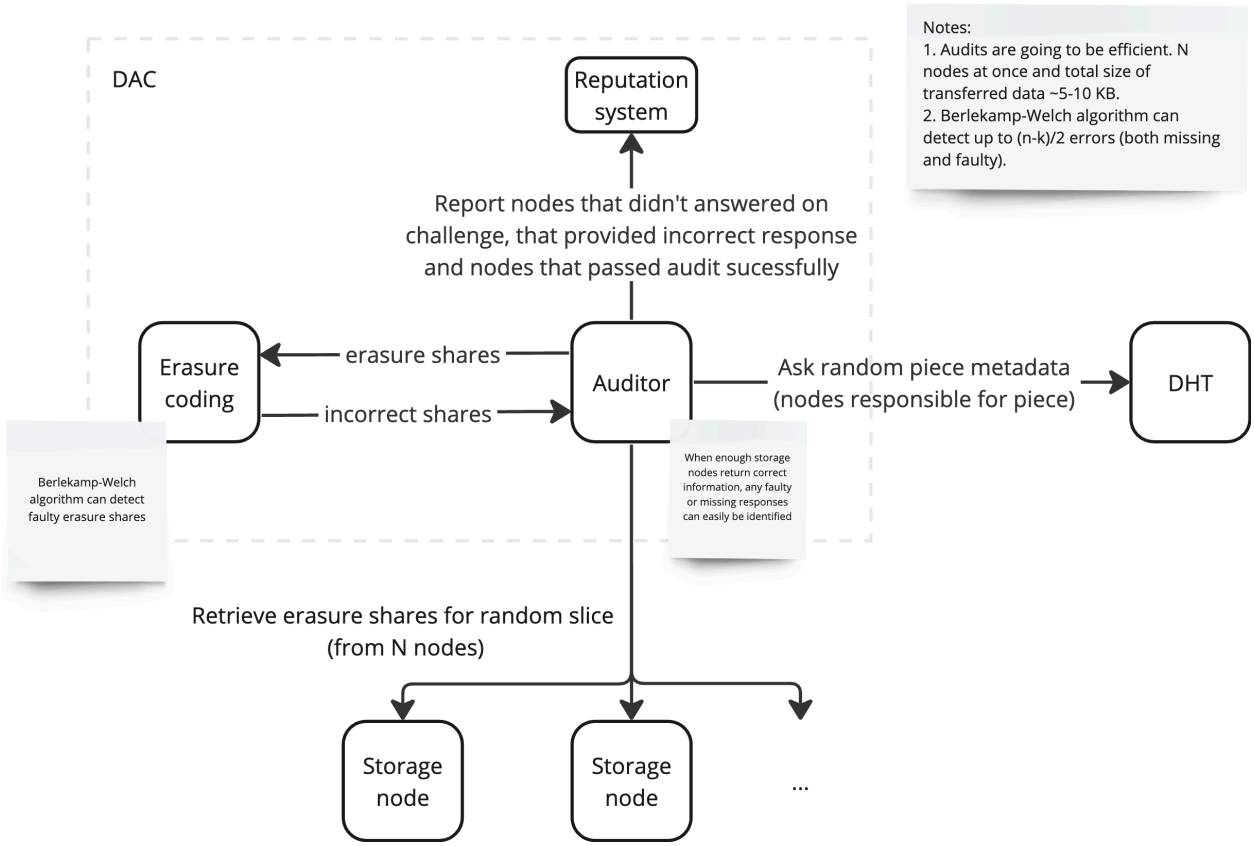
Slice is a small part (~2-5 KB) of data stored in DDC that can be used to optimise audits or range access.

Erasure share is a part on encoded slice.

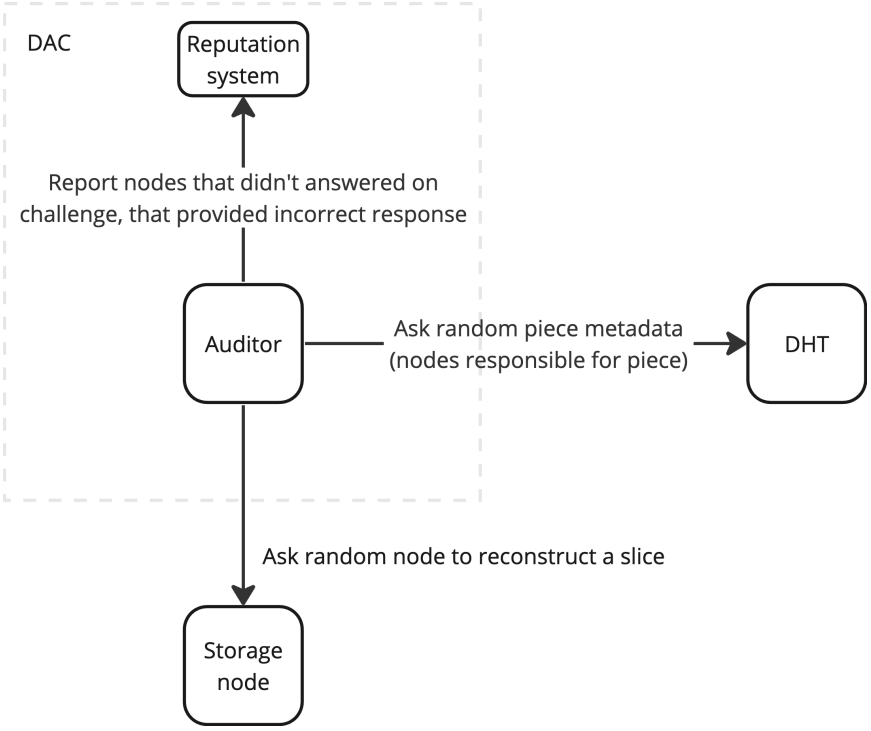


Audits and its effectiveness

Proof of storage (proofs that a storage node has a data it's responsible for)



Proof of retrievability (proofs that storage node is able to reconstruct a file from the network and return to client)



Redundancy savings (erasure coding vs replication)

The redundancy savings from using erasure coding instead of replication depends on the set of factors:

- expected durability level
- node churn rate
- *n* (or *k* in DHT) in replication scheme
- *k* and *n* in erasure coding scheme.
- optimisations that can be applied to erasure coding to avoid long tails and increase repairs efficiency (it's requires to generate more erasure shares to achieve the same level of durability).
- etc.

As far as those parameters are not fixed, savings can vary. In order to have at least an approximate understanding of the possible redundancy savings, I propose to take the following values:

- Cere DDC supposed to be a decentralised storage that provide enterprise level guarantees. Erasure coding is going to be applied on large pieces that

can be treated as objects (files). As a reference for durability guarantees AWS S3 can be used. They provide 99.999999999% (11 9th). So the same level is a minimum for Cere DDC.

- the fraction of nodes that will go offline in a month on average is a 10 percent
- *k = 20* in DHT (meaning that values stored in DHT are going to have 20 copies) as it's suggested in Kademlia paper
- data encoded via erasure coding and data stored in DHT should have the same durability level
- to simplify calculations additional parameters for erasure coding are skipped

Calculation method described in Storj whitepaper [page 21] is used.

The required level of durability via replication can be achieved with expansion factor of 17 (expansion factor of 20 give us 13 9th)

The required level of durability via erasure coding can be achieved via k = 16 and n = 40 with expansion factor of 2.5 (k = 16 n = 48 and expansion factor of 3 give us more than 13 9th).

Conclusion

Again, redundancy savings depend on several network parameters and they can be tuned over time based on network reliability and expected durability.

Example above demonstrated that erasure coding can be **~5 times more efficient** in terms of storage space required and network bandwidth.

Complexity comparison (erasure coding vs replication)

Obviously benefits provided by more advanced redundancy technique (erasure coding) comes at a cost. Erasure coding is more complicated than traditional replication and needs additional effort to implement:

1. Slicing, erasure coding and batching operations are straightforward. There are ready to use libraries that covers our needs. [EASY]
2. Audits should work on top of erasure coding (see Proof of storage). [EASY]
3. Additional optimisation to standard erasure coding can be added. For example threshold parameter (T) to avoid a need to repair each erasure share (wait

until there are T erasure shares lost). Or long tail parameter (L) to avoid performance issues because of some slow nodes out of N required [MEDIUM].

4. More complicated proofs should be designed to secure erasure coding process (see proof of durability and proof of erasure coding). Not original content (erasure shares instead of pieces) should be distributed across the network. Decentralisation phase 3. [MEDIUM or HARD]

Consequences

- More economically beneficial for both clients and storage providers (less physical space is used to achieve desired level of durability)
- Higher storage network capacity (because of the lower expansion factor, more data can be stored in the network)
- Higher network throughput (because less bandwidth used by data redundancy strategy)
- Lower latency. Piece erasure codes can be requested from **N** nodes (or some number higher that **K**) and first **K** answers are enough to rebuild a piece. Other slower requests can be cancelled.
- Efficient audits (affect a large number of nodes and small in size).
- Efficient data repairs.

Notes

- Erasure coding requires more metadata to be stored in DHT. In case metadata size is going to be as big as a piece, it's become inefficient and such pieces (e.g. <5 KB) should be stored directly in DHT and redundancy should be achieved via standard replication mechanism.